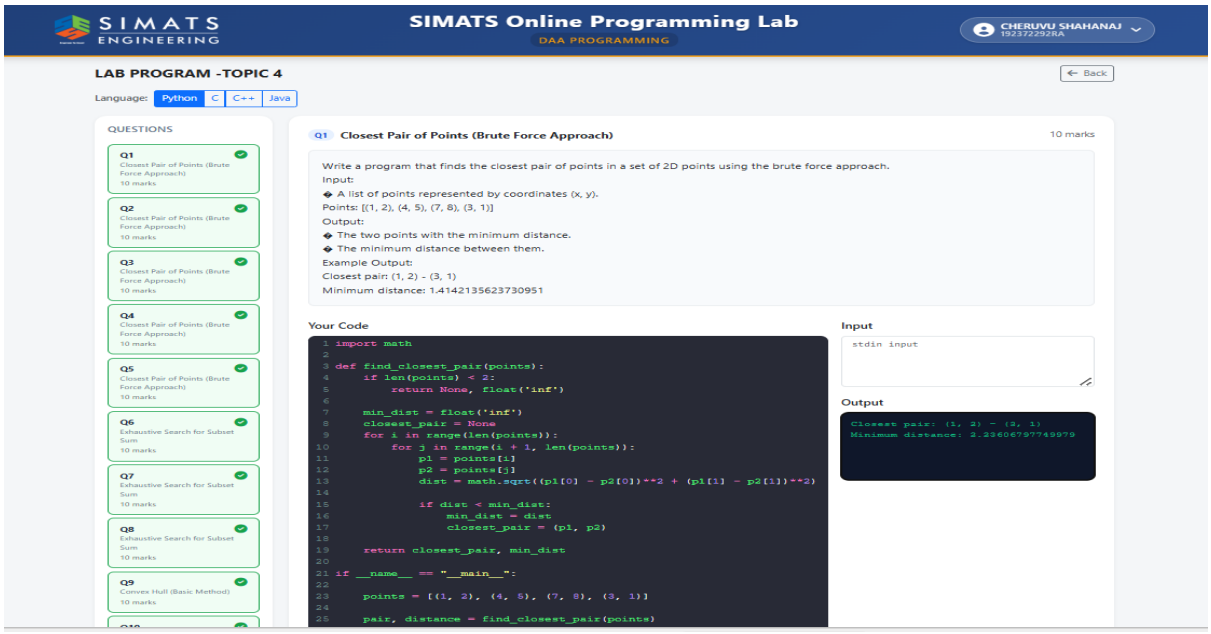
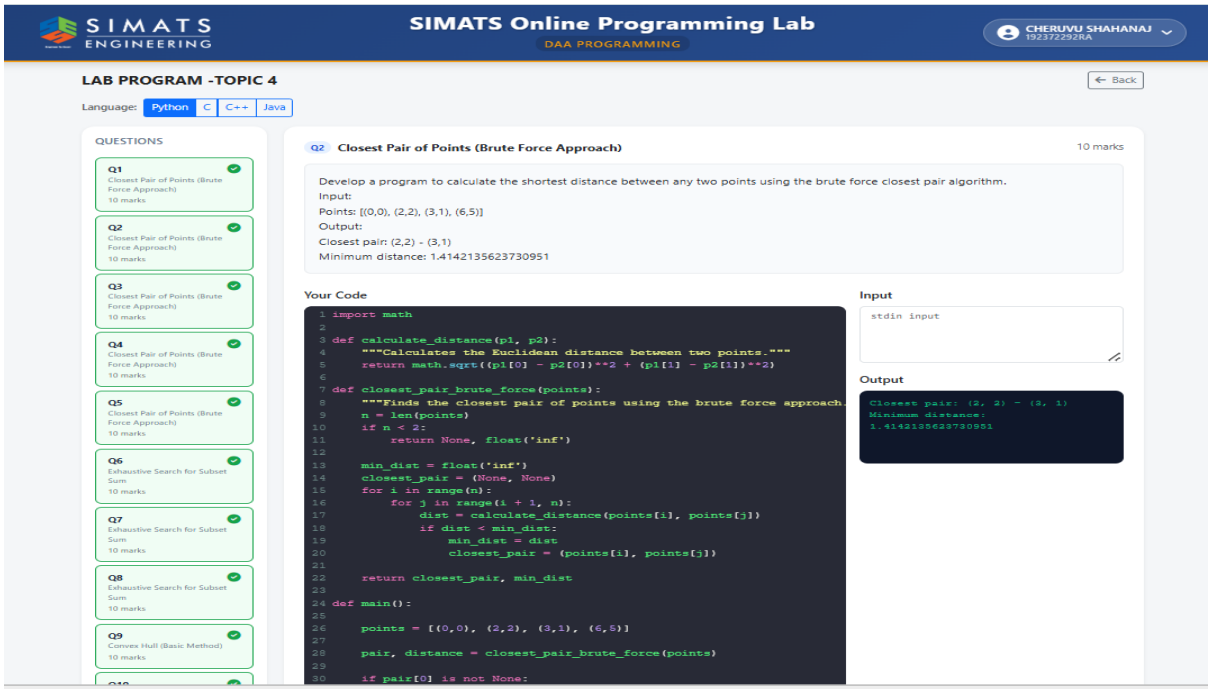
LAB PROGRAM - TOPIC 4

SHAHANAJ C

192372292





SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM -TOPIC 4

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Closest Pair of Points (Brute Force Approach) 10 marks
- Q2 Closest Pair of Points (Brute Force Approach) 10 marks
- Q3 Closest Pair of Points (Brute Force Approach) 10 marks
- Q4 Closest Pair of Points (Brute Force Approach) 10 marks
- Q5 Closest Pair of Points (Brute Force Approach) 10 marks
- Q6 Exhaustive Search for Subset Sum 10 marks
- Q7 Exhaustive Search for Subset Sum 10 marks
- Q8 Exhaustive Search for Subset Sum 10 marks
- Q9 Convex Hull (Basic Method) 10 marks

Q3 Closest Pair of Points (Brute Force Approach) — 10 marks

Write a program to identify the closest pair of points from a given collection of 2D points using brute force technique.
Input:
Points: [(10,20), (15,25), (30,40), (12,22)]
Output:
Closest pair: (10,20) - (12,22)
Minimum distance: 2.8284271247461903

Your Code

```python
import math

def closest_pair(points):
    n = len(points)
    min_dist = float('inf')
    closest = None

    for i in range(n):
        for j in range(i + 1, n):
            x1, y1 = points[i]
            x2, y2 = points[j]
            dist = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)

            if dist < min_dist:
                min_dist = dist
                closest = (points[i], points[j])

    return closest, min_dist
points = [(10, 20), (15, 25), (30, 40), (12, 22)]

pair, min_distance = closest_pair(points)

print(f"Closest pair: ({pair[0][0]},{pair[0][1]}) - ({pair[1][0]},{pai
print(f"Minimum distance: {min_distance}")
```

Input

```
stdin input
```

Output

```
Closest pair: (10,20) - (12,22)
Minimum distance:
2.8284271247461903
```

Submitted

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM -TOPIC 4

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Closest Pair of Points (Brute Force Approach) 10 marks
- Q2 Closest Pair of Points (Brute Force Approach) 10 marks
- Q3 Closest Pair of Points (Brute Force Approach) 10 marks
- Q4 Closest Pair of Points (Brute Force Approach) 10 marks
- Q5 Closest Pair of Points (Brute Force Approach) 10 marks
- Q6 Exhaustive Search for Subset Sum 10 marks
- Q7 Exhaustive Search for Subset Sum 10 marks
- Q8 Exhaustive Search for Subset Sum 10 marks
- Q9

Q4 Closest Pair of Points (Brute Force Approach) — 10 marks

Write a program to find the closest pair of points among the given set of coordinates using exhaustive comparison of all pairs.
Input:
Points: [(2, 3), (5, 4), (1, 7), (8, 9), (4, 6)]
Output:
Closest pair: (2, 3) - (5, 4)
Minimum distance: 3.1622776601683795

Your Code

```python
import math

points = [(2, 3), (5, 4), (1, 7), (8, 9), (4, 6)]

min_dist = float('inf')
closest_pair = None

n = len(points)
for i in range(n):
    for j in range(i + 1, n):
        x1, y1 = points[i]
        x2, y2 = points[j]
        dist = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)

        if dist < min_dist:
            min_dist = dist
            closest_pair = (points[i], points[j])

print("Closest pair:", closest_pair[0], "-", closest_pair[1])
print("Minimum distance:", min_dist)
```

Input

```
stdin input
```

Output

```
Closest pair: (5, 4) - (4, 6)
Minimum distance: 2.23606797749979
```

Submitted

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM -TOPIC 4

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Closest Pair of Points (Brute Force Approach) 10 marks
- Q2 Closest Pair of Points (Brute Force Approach) 10 marks
- Q3 Closest Pair of Points (Brute Force Approach) 10 marks
- Q4 Closest Pair of Points (Brute Force Approach) 10 marks
- Q5 Closest Pair of Points (Brute Force Approach) 10 marks
- Q6 Exhaustive Search for Subset Sum 10 marks
- Q7 Exhaustive Search for Subset Sum 10 marks
- Q8 Exhaustive Search for Subset Sum 10 marks
- Q9 Convex Hull (Basic Method) 10 marks

Q5 Closest Pair of Points (Brute Force Approach) — 10 marks

Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7(6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the same line?
Given points: P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6 (12.5, 7),
P7 (6, 6.5), P8 (7.5, 4.5).
output: P3, P4, P6, P5, P7, P1

Your Code

```python
import math

def calculate_distance(p1, p2):
    """Calculates the Euclidean distance between two points."""
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def find_closest_pair(points):
    """Finds the closest pair of points using the brute force approach.
    n = len(points)
    if n < 2:
        return None, float('inf')

    min_dist = float('inf')
    closest_pair = None
    for i in range(n):
        for j in range(i + 1, n):
            dist = calculate_distance(points[i], points[j])
            if dist < min_dist:
                min_dist = dist
                closest_pair = (points[i], points[j])

    return closest_pair, min_dist
if __name__ == "__main__":

    sample_points = [(1, 2), (4, 6), (1, 3), (10, 10), (5, 1)]

```

Input

```
stdin input
```

Output

```
The closest pair is: ((1, 2), (1, 3))
The minimum distance is: 1.0000
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

LAB PROGRAM -TOPIC 4

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Closest Pair of Points (Brute Force Approach) 10 marks
- Q2 Closest Pair of Points (Brute Force Approach) 10 marks
- Q3 Closest Pair of Points (Brute Force Approach) 10 marks
- Q4 Closest Pair of Points (Brute Force Approach) 10 marks
- Q5 Closest Pair of Points (Brute Force Approach) 10 marks
- Q6 Exhaustive Search for Subset Sum 10 marks
- Q7 Exhaustive Search for Subset Sum 10 marks
- Q8 Exhaustive Search for Subset Sum 10 marks
- Q9 Convex Hull (Basic Method) 10 marks

Q6 Exhaustive Search for Subset Sum — 10 marks

Write a program to find whether a subset exists with a given sum using exhaustive search technique.
Input:
Set: [3, 34, 4, 12, 5, 2]
Target Sum: 9
Output:
Subset found: [4,5]
Sum: 9

Your Code

```python
from itertools import combinations

def subset_sum_exhaustive(arr, target):
    n = len(arr)
    for r in range(1, n + 1):
        for subset in combinations(arr, r):
            if sum(subset) == target:
                return subset

    return None

arr = [3, 34, 4, 12, 5, 2]
target = 9

result = subset_sum_exhaustive(arr, target)

if result is not None:
    print("Subset found:", list(result))
    print("Sum:", target)
else:
    print("No subset found.")
```

Input

```
stdin input
```

Output

```
Subset found: [4, 5]
Sum: 9
```

Submitted

LAB PROGRAM -TOPIC 4

← Back

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Closest Pair of Points (Brute Force Approach) 10 marks
- Q2 Closest Pair of Points (Brute Force Approach) 10 marks
- Q3 Closest Pair of Points (Brute Force Approach) 10 marks
- Q4 Closest Pair of Points (Brute Force Approach) 10 marks
- Q5 Closest Pair of Points (Brute Force Approach) 10 marks
- Q6 Exhaustive Search for Subset Sum 10 marks
- Q7 Exhaustive Search for Subset Sum 10 marks
- Q8 Exhaustive Search for Subset Sum 10 marks
- Q9 Convex Hull (Basic Method) 10 marks
- Q10

Q7 Exhaustive Search for Subset Sum — 10 marks

Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Test Cases:

1. Simple Case: Four cities with basic coordinates (e.g., [(1, 2), (4, 5), (7, 1), (3, 6)])
2. More Complex Case: Five cities with more intricate coordinates (e.g., [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)])

Output:

Test Case 1:

Shortest Distance: 7.0710678118654755

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 14.142135623730951

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]

Your Code

```python
import itertools
import math

def calculate_distance(p1, p2):
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def solve_tsp_exhaustive(cities):
    num_cities = len(cities)
    if num_cities == 0:
        return 0, []
    start_city = cities[0]
    other_cities = cities[1:]

    min_distance = float('inf')
    best_path = []
    for permutation in itertools.permutations(other_cities):
        current_path = [start_city] + list(permutation) + [start_city]
        current_distance = 0

        for i in range(len(current_path) - 1):
            current_distance += calculate_distance(current_path[i], cu
```

Input

```
stdin input
```

Output

```
Test Case 1:
Shortest Distance:
16.969112047670894
Shortest Path: [(1, 2), (7, 1),
(4, 5), (3, 6), (1, 2)]
```


- Q2 Closest Pair of Points (Brute Force Approach) 10 marks
- Q3 Closest Pair of Points (Brute Force Approach) 10 marks
- Q4 Closest Pair of Points (Brute Force Approach) 10 marks
- Q5 Closest Pair of Points (Brute Force Approach) 10 marks
- Q6 Exhaustive Search for Subset Sum 10 marks
- Q7 Exhaustive Search for Subset Sum 10 marks
- Q8 Exhaustive Search for Subset Sum 10 marks
- Q9 Convex Hull (Basic Method) 10 marks
- Q10 Convex Hull (Basic Method) 10 marks

10/10 answered

cost by summing the corresponding costs from the cost matrix Implement a function assignment_problem(cost_matrix) that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).

Test Cases:

Input

1. Simple Case: Cost Matrix: [[3, 10, 7], [8, 5, 12], [4, 6, 9]]
2. More Complex Case: Cost Matrix: [[15, 9, 4], [8, 7, 18], [6, 12, 11]]

Output:

Test Case 1:

Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker 3, task 3)]

Total Cost: 19

Test Case 2:

Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker 3, task 2)]

Total Cost: 24

Your Code

```python
import itertools

def total_cost(assignment, cost_matrix):

    current_total = 0
    for worker_idx, task_idx in enumerate(assignment):
        current_total += cost_matrix[worker_idx][task_idx]
    return current_total

def assignment_problem(cost_matrix):

    num_workers = len(cost_matrix)
    num_tasks = len(cost_matrix[0])

    task_indices = list(range(num_tasks))

    min_cost = float('inf')
    best_assignment_indices = None

    for p in itertools.permutations(task_indices):

        current_cost = total_cost(p, cost_matrix)

        if current_cost < min_cost:
            min_cost = current_cost
            best_assignment_indices = p


    formatted_assignment = []
    for i, task_idx in enumerate(best_assignment_indices):
```

Input

```
stdin input
```

Output

```
Test Case 1:
Optimal Assignment: [(worker 1,
task 2), (worker 2, task 2),
(worker 3, task 1)]
Total Cost: 16
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Closest Pair of Points (Brute Force Approach) 10 marks
- Q2 Closest Pair of Points (Brute Force Approach) 10 marks
- Q3 Closest Pair of Points (Brute Force Approach) 10 marks
- Q4 Closest Pair of Points (Brute Force Approach) 10 marks
- Q5 Closest Pair of Points (Brute Force Approach) 10 marks
- Q6 Exhaustive Search for Subset Sum 10 marks
- Q7 Exhaustive Search for Subset Sum 10 marks
- Q8 Exhaustive Search for Subset Sum 10 marks
- Q9 Convex Hull (Basic Method) 10 marks
- Q10 Convex Hull (Basic Method) 10 marks

Q9 Convex Hull (Basic Method) — 10 marks

Develop a program to find the outer boundary of a set of points using the convex hull algorithm.
Input:
Points: [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]
Output:
Convex Hull Points:
(4,1), (6,3), (5,5), (3,6), (1,4), (2,2)

Your Code

```
def cross(o, a, b):

    return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0

def convex_hull(points):

    points = sorted(set(points))
    if len(points) <= 1:
        return points

    lower = []
    for p in points:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0:
            lower.pop()
        lower.append(p)

    upper = []
    for p in reversed(points):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0:
            upper.pop()
        upper.append(p)

    return lower[:-1] + upper[:-1]

pts = [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]

hull = convex_hull(pts)
print("Convex Hull Points:")
for p in hull:
    print(p)
```

Input

```
stdin input
```

Output

```
Convex Hull Points:
(1, 4)
(2, 2)
(4, 1)
(6, 3)
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

CHERUVU SHAHANAJ
192372292RA

Language: Python | C | C++ | Java

QUESTIONS

- Q1 Closest Pair of Points (Brute Force Approach) 10 marks
- Q2 Closest Pair of Points (Brute Force Approach) 10 marks
- Q3 Closest Pair of Points (Brute Force Approach) 10 marks
- Q4 Closest Pair of Points (Brute Force Approach) 10 marks
- Q5 Closest Pair of Points (Brute Force Approach) 10 marks
- Q6 Exhaustive Search for Subset Sum 10 marks
- Q7 Exhaustive Search for Subset Sum 10 marks
- Q8 Exhaustive Search for Subset Sum 10 marks
- Q9 Convex Hull (Basic Method) 10 marks
- Q10

Q10 Convex Hull (Basic Method) — 10 marks

Write a program to calculate the convex hull of given points and display the points forming the polygon boundary.
Input:
Points: [(1,1), (2,4), (5,5), (6,2), (3,0)]
Output:
Convex Hull Points:
(3,0), (6,2), (5,5), (2,4)

Your Code

```
def convex_hull(points):
    points.sort()
    if len(points) <= 2:
        return points
    def cross_product(o, a, b):
        return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] -

    lower = []
    for p in points:
        while len(lower) >= 2 and cross_product(lower[-2], lower[-1], p
            lower.pop()
        lower.append(p)

    upper = []
    for p in reversed(points):
        while len(upper) >= 2 and cross_product(upper[-2], upper[-1], p
            upper.pop()
        upper.append(p)
    return lower[:-1] + upper[:-1]
points = [(1, 1), (2, 4), (5, 5), (6, 2), (3, 0)]
hull = convex_hull(points)
print("Convex Hull Points:")
print(", ".join(map(str, hull)))
```

Input

```
stdin input
```

Output

```
Convex Hull Points:
(1, 1), (3, 0), (6, 2), (5, 5), (2,
4)
```

Submitted